



University
of Exeter

ECM2433 Sample

The C Family

On Campus Exam

Closed Book

Additional Materials:
None

Permitted Materials:
No Calculators Permitted

Guidelines:

- The duration of this paper is 2hr 00**
- Word Count: No Word Count**
- Write all answers in the answer booklet provided.**
- Do not open the exam paper until instructed by the Invigilator.**
- Use only black pen or pencil.**
- Mobile and electronic devices are not allowed.**
- Do not communicate with other candidates during the exam.**

Exam Instructions:

Answer ALL questions.

The marks for this module are calculated from 50% of the percentage mark for this paper plus 50% of the percentage mark for associated coursework.

This is a sample paper in the style
of the May paper.

Possible answers are *skeleton*
answers.

Question 1

- (a) A programmer uses the following macro construction to find the square of two numbers:

```
1 #define square(x) x * x
```

Using this macro, what is the result of running the following code?

```
1 int k = 8;  
2 printf("%d\n", square(k+1));
```

Explain how the macro might be modified to yield the square of $k+1$.

(4 marks)

The macro is expanded to $k + 1 * k + 1$ which is $2*k+1$ or 17. Defining

```
1 #define square(x) (x) * (x)
```

will force the evaluation of $(k + 1)^2$.

Give an advantage and a disadvantage of this method compared with defining an equivalent function.

(4 marks)

Advantage: Faster as no function call is involved.

Disadvantage: Potential side effects, for example if an argument is incremented.

- (b) Give the definition of a C preprocessor macro `swap(a, b)` which interchanges its `int` arguments.

(4 marks)

```
1 #define swap(a, b) { int tmp = a; a = b; b = tmp; }
```

Other answers involving addition and subtraction or exclusive or are possible.

- (c) Given the declarations:

```
1 char str[] = "By Divine Aid";  
2 char *dup, *p, *q;
```

write code to allocate memory and then copy `str` into `dup`. You may use `malloc`, but no other variables or functions.

(8 marks)

```

1      #include <assert.h>
2
3      for (p = str; *p; p++)
4      ;
5
6      dup = (char *)malloc(sizeof(char)*(p-str)+1);
7      assert(dup != NULL);
8
9      for (p = str, q = dup; *p; )
10     *q++ = *p++;
11     *q = '\0'

```

Marks for finding the length of the string with pointer arithmetic, for cast [1], error check, loops and pointer incrementing

- (d) Write a function `int **alloc2d(int rows, int cols)` to allocate a two-dimensional array of integers.

(6 marks)

```

1      #include <stdlib.h>
2      #include <assert.h>
3
4      int **alloc2d(int rows, int cols)
5      {
6          int **a = (int **)malloc(rows*sizeof(int *));
7          assert(a != NULL);
8          a[0] = (int *)malloc(row*cols*sizeof(int));
9          assert(a[0] != NULL);
10
11         for (int r = 1; r < rows; ++r)
12             a[r] = a[0] + r*rows;
13         return a;
14     }

```

Answers that allocate rows separately are fine.

Marks for allocation of `a`; allocation of `rows * cols * sizeof(int)`; pointer arithmetic or separate allocation appreciating that each row is a pointer to `cols` ints; checking that `malloc` has not returned a null pointer.

- (e) Explain the difference between *global* and *static* variables in C.

(5 marks)

- global variables have global scope and are accessible everywhere.
- static variables have scope limited to the function in which they are declared but persist between invocations of the function.

(f) A programmer writes the following code

```
1 int i;  
2 for (i = 0; i < 10; i++)  
3     printf("%d ", i);  
4 sleep(10);  
5 exit(0);
```

but is surprised to see no output until the program exits. Explain why and how the result of each `printf` can be seen immediately.

(6 marks)

`printf` makes a buffered write; output is stored in a buffer until the buffer is full or an end-of-line is reached (if connected to a tty) or the program exits normally at which point the buffer is actually written.

Buffer can be flushed immediately with `fflush(stdout)`.

(Total 37 marks)

Question 2

A class is defined as follows:

```
1 class ClockType
2 {
3     unsigned int hours;
4     unsigned int minutes;
5     unsigned int seconds;
6
7     protected:
8     char *maker;
9
10    public:
11    unsigned int seconds_since_midnight()
12    {
13        return seconds + minutes*60 + hours*60*60;
14    }
15 };
```

- (a) Give code to derive a class, Watch, from the ClockType base class. Your Watch class should have an additional member function printMaker that prints the maker string.

(4 marks)

Possible code is:

```
class Watch: public ClockType
{
    public:
    void printMaker()
    {
        std::cout << maker << std::endl;
    }
};
```

Marks for inheritance and for printMaker function.

- (b) Explain why the Watch class can access the maker attribute, but not the hours, minutes and seconds variables.

(4 marks)

The maker attribute is protected [1] and therefore accessible to children of ClockType, but hours, minutes and seconds are private [1] (because not declared public or protected).

- (c) Why might the designer of the ClockType class prefer to make the hours, minutes and seconds variables inaccessible to derived classes?

(4 marks)

Making them private allows the designer to define an interface [2] keeping the implementation hidden so it could be subsequently changed without troubling users of the class.

Should be well understood, but many students know the mechanisms but not the reasons for using them.

- (d) The programmer of another class wishes to obtain and change the `hours`, `minutes` and `seconds` variables. Suggest **two** mechanisms that could be used to accomplish this.

(4 marks)

Possible mechanisms are:

- Declare functions accessing the variables to be friends.
- Provided getter and setter functions in `ClockType`.

Only 1 mark for making everything public.

- (e) A further class, `SmartWatch`, is derived from the `Watch` class with access modifier `public`. State the access control of each of the variables and member functions of the `ClockType` class in the `SmartWatch` class.

(4 marks)

- `hours`, `minutes`, `seconds` are not accessible as private in `ClockType`.
- `maker` is accessible because protected in `ClockType` and it is accessible in `Watch`.
- `seconds_since_midnight` is public in `ClockType`, `Watch` and `SmartWatch`.

(Total 20 marks)

Question 3

(a) Consider the following Rust code:

```
1 fn main() {  
2     let s1 = String::from("hello");  
3     let s2 = s1;  
4     println!("{}", s1);  
5 }
```

(i) Explain why this code fails to compile.

(2 marks)

The code fails because ownership of the string is moved from `s1` to `s2` when the assignment `let s2 = s1` is made.

(ii) How does Rust's approach to memory management here differ from copying a pointer in C?

(2 marks)

In C, copying a pointer would result in a shallow copy where both pointers refer to the same memory location. In Rust, ownership rules strictly enforce that a value can only have one owner at a time, preventing issues like double frees and dangling pointers.

(b) Consider the following Rust code:

```
1 fn main() {  
2     let x: i32 = 10;  
3     let y = x;  
4  
5     let s1 = String::from("hi");  
6     let s2 = s1;  
7  
8     println!("{x} {y} {s1} {s2}");  
9 }
```

(i) State which assignment performs a copy and which performs a move. Justify your answer.

(2 marks)

The assignment `let y = x;` performs a copy because `i32` is a primitive type that implements the `Copy` trait. The assignment `let s2 = s1;` performs a move because `String` is a heap-allocated type that does not implement the `Copy` trait.

(ii) Explain why the program does not compile.

(2 marks)

The program does not compile because after the move, `s1` is no longer valid, and attempting to use it in the `println!` macro results in a compile-time error.

- (iii) Suggest a modification to the code that would allow it to compile successfully while preserving the printing of the four values.

(2 marks)

To allow the code to compile successfully, we can clone `s1` before moving it to `s2`: `let s2 = s1.clone();` This way, both `s1` and `s2` will be valid for printing.

- (c) In C, iterating over an array typically requires manual index management:

```
1 int arr[] = {1, 2, 3, 4, 5};
2 for (int i = 0; i < 5; i++) {
3     printf("%d\n", arr[i]);
4 }
```

In Rust, the same iteration can be written as:

```
1 let arr = [1, 2, 3, 4, 5];
2 for x in &arr {
3     println!("{}", x);
4 }
```

- (i) Give two potential errors that can occur with C's manual indexing approach that Rust's iterator approach prevents.

(4 marks)

Off-by-one errors (e.g., using `<=` instead of `<`, or starting at 1 instead of 0).

Out-of-bounds access if the length constant (5) doesn't match the actual array size.

- (ii) What is the type of `x` in the Rust loop above?

(1 mark)

`&i32` (a reference to `i32`).

- (iii) Using iterator methods, write a single expression that takes a `Vec<i32>` called `numbers` and produces a new `Vec<i32>` containing only the even numbers, each multiplied by 2.

(4 marks)

A possible expression is:

```
1 let result: Vec<i32> = numbers.into_iter()  
2   .filter(|x| x % 2 == 0)  
3   .map(|x| x * 2)  
4   .collect();
```

Marks for `filter` with correct even check, for `map` with doubling, for `collect`, for correct type annotation (or `turbofish`). Also accept `.iter()` with appropriate dereferencing.

(Total 19 marks)

Question 4

(a) Memory management is handled differently in C, C++, and Rust.

- (i) In C, heap memory is allocated using `malloc` and released using `free`. Give two errors that can occur if `free` is used incorrectly, and explain the consequences of each.

(3 marks)

Double free: calling `free` twice on the same pointer can corrupt the heap allocator's internal data structures, leading to crashes or security vulnerabilities.

Use-after-free (dangling pointer): accessing memory after it has been freed leads to undefined behaviour; the memory may have been reallocated for another purpose.

- (ii) C++ introduces the RAII (Resource Acquisition Is Initialization) pattern. Explain what RAII means and how it helps prevent the errors you described above.

(4 marks)

RAII ties the lifetime of a resource (such as heap memory) to the lifetime of an object. Resources are acquired in the constructor and released in the destructor. When the object goes out of scope, the destructor is automatically called, ensuring resources are freed exactly once. This prevents both double-free (destructor runs once) and use-after-free (object no longer accessible after destruction).

- (iii) Rust does not have constructors or destructors in the C++ sense, yet it also guarantees memory safety without a garbage collector. Explain how Rust's ownership system achieves this, referring to the three ownership rules.

(5 marks)

Rust's three ownership rules are:

1. Each value in Rust has a variable that is its owner
2. There can only be one owner at a time
3. When the owner goes out of scope, the value is dropped (memory freed)

These rules are enforced at compile time by the borrow checker. Because ownership is tracked statically, the compiler knows exactly when to insert deallocation code (calling the `Drop` trait if implemented), preventing double-free and use-after-free errors.

- (iv) C++ provides `std::unique_ptr` for unique ownership and `std::shared_ptr` for shared ownership. What are the analogous types in Rust, and what key

difference exists in how Rust enforces their correct usage?

(3 marks)

`Box<T>` is analogous to `std::unique_ptr` (unique ownership), and `Rc<T>` is analogous to `std::shared_ptr` (reference-counted shared ownership).

The key difference is that Rust enforces ownership rules at compile time, whereas C++ smart pointers rely on runtime mechanisms and programmer discipline.

(b) Error handling is approached differently in C, C++, and Rust.

(i) In C, a common pattern for error handling is shown below:

```
1 FILE *f = fopen("data.txt", "r");
2 if (f == NULL) {
3     // handle error
4 }
```

Give two disadvantages of this return-code approach to error handling.

(3 marks)

The programmer can forget to check the return value, leading to bugs when the error case is silently ignored.

The return value is overloaded: a single type (e.g., pointer or integer) must represent both success values and error indicators, which can be confusing and error-prone.

(Also accept: error information is limited; `errno` is global and can be overwritten; cannot return both a value and rich error information.)

(ii) C++ introduces exceptions for error handling. Give one advantage and one disadvantage of exceptions compared to C's return-code approach.

(3 marks)

Advantage: Exceptions cannot be silently ignored; if not caught, they propagate up the call stack and terminate the program, making errors visible.

(Also accept: exceptions can carry rich error information; they separate error-handling code from normal code flow)

Disadvantage: Exceptions have runtime overhead; they can make control flow harder to follow; they require careful handling to ensure resources are cleaned up (exception safety).

(Also accept: exceptions can be thrown from unexpected places; difficulty reasoning about which functions can throw)

- (iii) Rust uses `Result<T, E>` instead of exceptions. The following Rust function reads a file:

```
1 use std::fs::File;
2 use std::io::{self, Read};
3
4 fn read_file(path: &str) -> Result<String, io::Error> {
5     let mut file = File::open(path)?;
6     let mut contents = String::new();
7     file.read_to_string(&mut contents)?;
8     Ok(contents)
9 }
```

Explain how Rust's approach combines advantages of both C's explicitness and C++'s automatic propagation, referring to the `?` operator.

(3 marks)

Like C, errors are explicit in the return type; the function signature clearly shows it can fail with `io::Error`, making error cases visible in the API.

Unlike C, the compiler warns if a `Result` is unused, preventing accidental ignoring of errors.

The `?` operator provides automatic propagation similar to exceptions: if an operation returns `Err`, it immediately returns from the function with that error.

However, unlike exceptions, the `?` operator only works in functions that return `Result`, making error propagation paths explicit and visible in the type system.

(Total 24 marks)